

This technique is called *w-buffering*, because the view-space  $z$ -coordinate conveniently appears in the  $w$ -component of our homogeneous clip-space coordinates. (Recall from Equation (11.1) that  $p_{H_w} = -p_{V_z}$ .)

The terminology can be very confusing here. The  $z$ - and  $w$ -buffers store coordinates that are expressed in *clip space*. But in terms of *view-space* coordinates, the  $z$ -buffer stores  $1/z$  (i.e.,  $1/p_{V_z}$ ) while the  $w$ -buffer stores  $z$  (i.e.,  $p_{V_z}$ )!

We should note here that the *w-buffering* approach is a bit more expensive than its  $z$ -based counterpart. This is because with *w-buffering*, we cannot linearly interpolate depths directly. Depths must be inverted prior to interpolation and then re-inverted prior to being stored in the  $w$ -buffer.

## 11.2 The Rendering Pipeline

Now that we've completed our whirlwind tour of the major theoretical and practical underpinnings of triangle rasterization, let's turn our attention to how it is typically implemented. In real-time game rendering engines, the high-level rendering steps described in Section 11.1 are implemented using a software/hardware architecture known as a *pipeline*. A pipeline is just an ordered chain of computational stages, each with a specific purpose, operating on a stream of input data items and producing a stream of output data.

Each stage of a pipeline can typically operate independently of the other stages. Hence, one of the biggest advantages of a pipelined architecture is that it lends itself extremely well to parallelization. While the first stage is chewing on one data element, the second stage can be processing the results previously produced by the first stage, and so on down the chain.

Parallelization can also be achieved within an individual stage of the pipeline. For example, if the computing hardware for a particular stage is duplicated  $N$  times on the die,  $N$  data elements can be processed in parallel by that stage. A parallelized pipeline is shown in Figure 11.40. Ideally the stages operate in parallel (most of the time), and certain stages are capable of operating on multiple data items simultaneously as well.

The *throughput* of a pipeline measures how many data items are processed per second overall. The pipeline's *latency* measures the amount of time it takes for a single data element to make it through the entire pipeline. The latency of an individual stage measures how long that stage takes to process a single item. The slowest stage of a pipeline dictates the throughput of the entire pipeline. It also has an impact on the average latency of the pipeline as a whole. Therefore, when designing a rendering pipeline, we attempt to minimize and balance latency across the entire pipeline and eliminate bottlenecks. In a well-

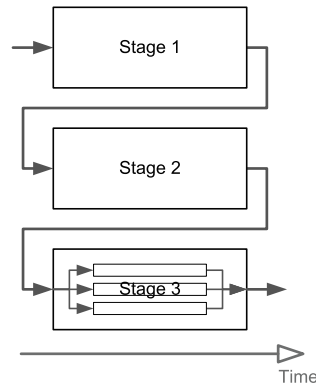


Figure II.40. A parallelized pipeline. The stages all operate in parallel, and some stages are capable of operating on multiple data items simultaneously as well.

designed pipeline, all the stages operate simultaneously, and no stage is ever idle for very long waiting for another stage to become free.

### II.2.1 Overview of the Rendering Pipeline

Some graphics texts divide the rendering pipeline into three coarse-grained stages. In this book, we'll extend this pipeline back even further, to encompass the offline tools used to create the scenes that are ultimately rendered by the game engine. The high-level stages in our pipeline are:

- *Tools stage (offline)*. Geometry and surface properties (materials) are defined.
- *Asset conditioning stage (offline)*. The geometry and material data are processed by the asset conditioning pipeline (ACP) into an engine-ready format.
- *Application stage (CPU)*. Potentially visible mesh instances are identified and submitted to the graphics hardware along with their materials for rendering.
- *Geometry processing stage (GPU)*. Vertices are transformed and lit and projected into homogeneous clip space. Triangles are processed by the optional geometry shader and then clipped to the frustum.
- *Rasterization stage (GPU)*. Triangles are converted into fragments that are shaded, passed through various tests (z-test, alpha test, stencil test, etc.) and finally blended into the frame buffer.

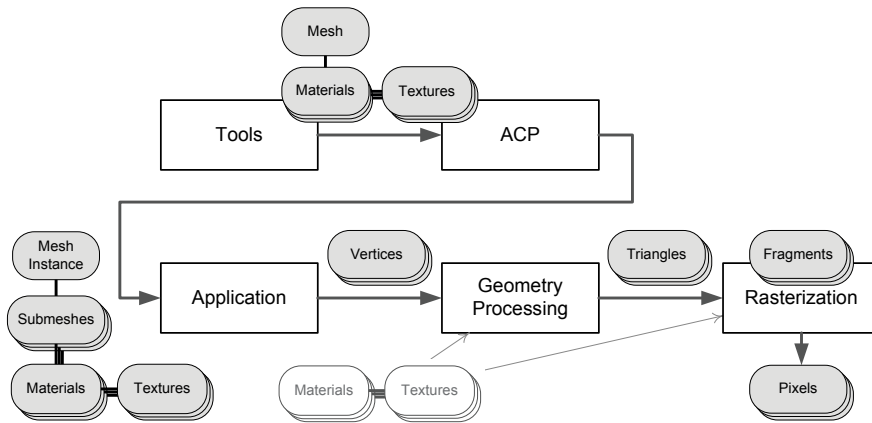


Figure 11.41. The format of geometric data changes radically as it passes through the various stages of the rendering pipeline.

### 11.2.1.1 How the Rendering Pipeline Transforms Data

It's interesting to note how the format of geometry data changes as it passes through the rendering pipeline. The tools and asset conditioning stages deal with meshes and materials. The application stage deals in terms of mesh instances and submeshes, each of which is associated with a single material. During the geometry stage, each submesh is broken down into individual vertices, which are processed largely in parallel. At the conclusion of this stage, the triangles are reconstructed from the fully transformed and shaded vertices. In the rasterization stage, each triangle is broken into fragments, and these fragments are either discarded, or they are eventually written into the frame buffer as colors. This process is illustrated in Figure 11.41.

### 11.2.1.2 Implementation of the Pipeline

The first two stages of the rendering pipeline are implemented offline, usually executed by a Windows or Linux machine. The application stage is typically run on one or more CPU cores, whereas the geometry and rasterization stages are usually executed by the graphics processing unit (GPU). In the following sections, we'll explore some of the details of how each of these stages is implemented.

## 11.2.2 The Tools Stage

In the tools stage, meshes are authored by 3D modelers in a *digital content creation* (DCC) application like Maya, 3ds Max, Lightwave, Softimage/XSI,

SketchUp, etc. The models may be defined using any convenient surface description—NURBS, quads, triangles, etc. However, they are invariably tessellated into triangles prior to rendering by the runtime portion of the pipeline.

The vertices of a mesh may also be skinned. This involves associating each vertex with one or more joints in an articulated skeletal structure, along with weights describing each joint's relative influence over the vertex. Skinning information and the skeleton are used by the animation system to drive the movements of a model—see Chapter 12 for more details.

Materials are also defined by the artists during the tools stage. This involves selecting a shader for each material, selecting textures as required by the shader, and specifying the configuration parameters and options of each shader. Textures are mapped onto the surfaces, and other vertex attributes are also defined, often by “painting” them with some kind of intuitive tool within the DCC application.

Materials are usually authored using a commercial or custom in-house material editor. The material editor is sometimes integrated directly into the DCC application as a plug-in, or it may be a stand-alone program. Some material editors are live-linked to the game, so that material authors can see what the materials will look like in the real game. Other editors provide an offline 3D visualization view. Some editors even allow shader programs to be written and debugged by the artist or a shader engineer. Such tools allow rapid prototyping of visual effects by connecting various kinds of nodes together with a mouse. These tools generally provide a WYSIWYG display of the resulting material. NVIDIA's Fx Composer is an example of such a tool. Sadly, NVIDIA is no longer updating Fx Composer, and it only supports shader models up to DirectX 10. But they do offer a new Visual Studio plugin called NVIDIA® Nsight™ Visual Studio Edition. Depicted in Figure 11.42, Nsight provides powerful shader authoring and debugging facilities. The Unreal Engine also provides a graphical shader editor called Material Editor; it is shown in Figure 11.43.

Materials may be stored and managed with the individual meshes. However, this can lead to duplication of data—and effort. In many games, a relatively small number of materials can be used to define a wide range of objects in the game. For example, we might define some standard, reusable materials like wood, rock, metal, plastic, cloth, skin and so on. There's no reason to duplicate these materials inside every mesh. Instead, many game teams build up a library of materials from which to choose, and the individual meshes refer to the materials in a loosely coupled manner.



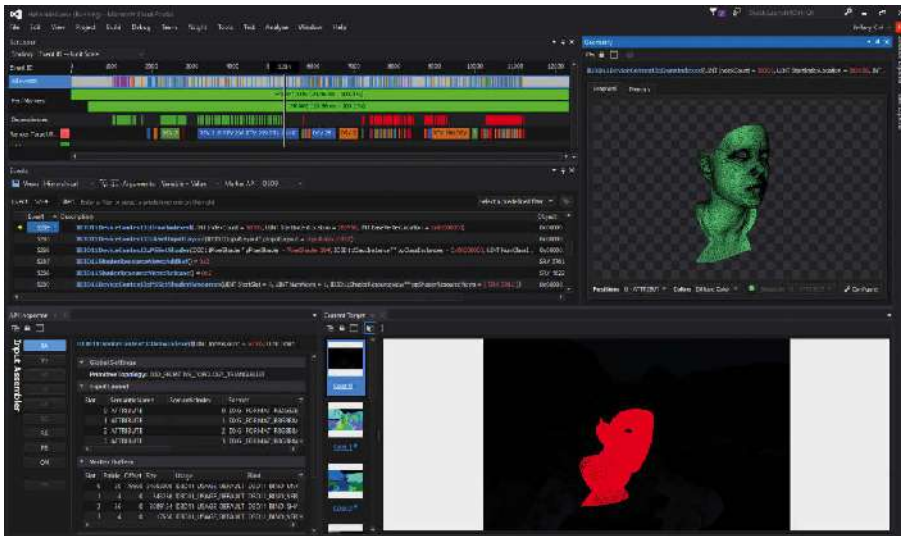


Figure 11.42. NVIDIA® Nsight™ Visual Studio Edition allows shader programs to be written, previalized and debugged easily.

### 11.2.3 The Asset Conditioning Stage

The asset conditioning stage is itself a pipeline, sometimes called the *asset conditioning pipeline* (ACP) or the *tools pipeline*. As we saw in Section 7.2.1.4, its job is to export, process and link together multiple types of assets into a cohesive whole. For example, a 3D model is comprised of geometry (vertex and index buffers), materials, textures and an optional skeleton. The ACP ensures that all of the individual assets referenced by a 3D model are available and ready to be loaded by the engine.

Geometric and material data is extracted from the DCC application and is usually stored in a platform-independent intermediate format. The data is then further processed into one or more platform-specific formats, depending on how many target platforms the engine supports. Ideally the platform-specific assets produced by this stage are ready to load into memory and use with little or no postprocessing at runtime. For example, mesh data targeted for the Xbox One or PS4 might be output as index and vertex buffers that are ready to be consumed by the GPU; on the PS3, geometry might be produced in compressed data streams that are ready to be DMA'd to the SPUs for decompression. The ACP often takes the needs of the material/shader into account

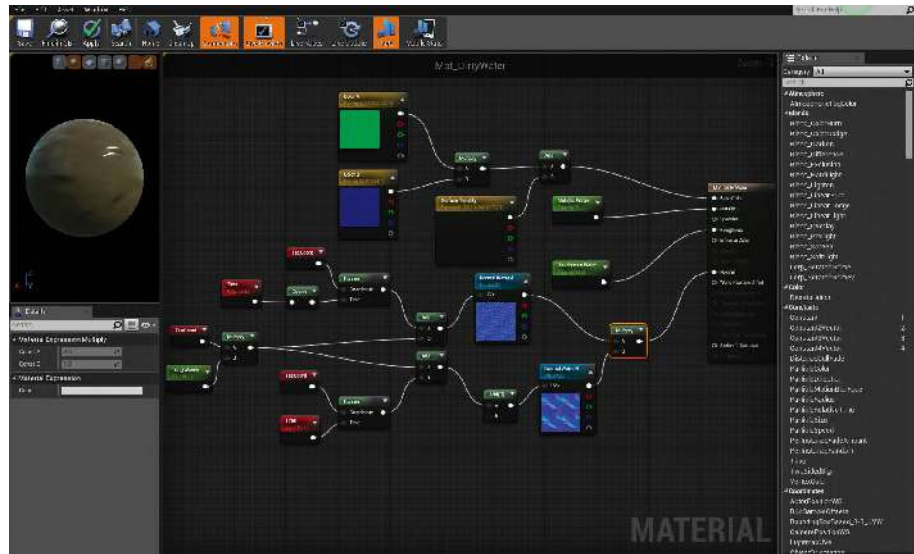


Figure 11.43. The Unreal Engine 4 Material Editor.

when building assets. For example, a particular shader might require tangent and bitangent vectors as well as a vertex normal; the ACP could generate these vectors automatically.

High-level *scene graph* data structures may also be computed during the asset conditioning stage. For example, static-level geometry may be processed in order to build a BSP tree. (As we'll investigate in Section 11.2.7.4, scene graph data structures help the rendering engine to very quickly determine which objects should be rendered, given a particular camera position and orientation.)

Expensive lighting calculations are often done offline as part of the asset conditioning stage. This is called *static lighting*; it may include calculation of light colors at the vertices of a mesh (this is called “baked” vertex lighting), construction of texture maps that encode per-pixel lighting information known as *light maps*, calculation of *precomputed radiance transfer* (PRT) coefficients (usually represented by spherical harmonic functions) and so on.

## 11.2.4 The GPU Pipeline

Graphics hardware has evolved around a specialized type of microprocessor known as a *graphics processing unit* or GPU. As we discussed in Section 4.11,

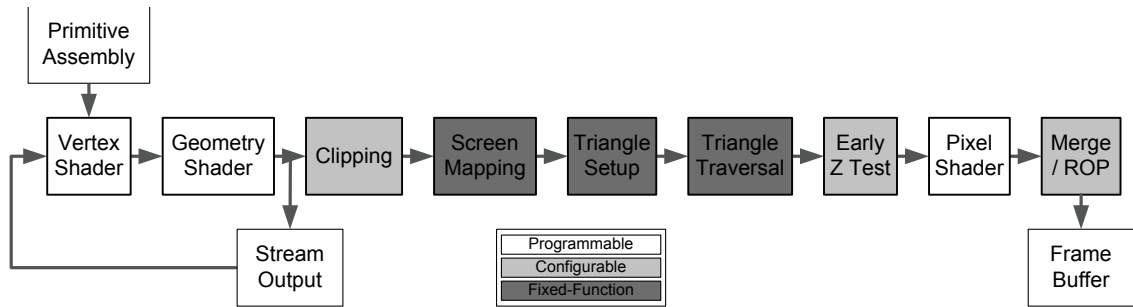


Figure 11.44. The geometry processing and rasterization stages of the rendering pipeline, as implemented by a typical GPU. The white stages are programmable, the light grey stages are configurable, and the dark grey boxes are fixed-function.

a GPU is designed to maximize throughput of the graphics pipeline, which it achieves through massive parallelization of tasks like vertex processing and per-pixel shading calculations. For example, a modern GPU like the AMD Radeon™ 7970 can achieve a peak performance of 4 TFLOPS, which it does by executing workloads in parallel across 32 compute units, each of which contains four 16-lane SIMD VPUs, which in turn execute pipelined wavefronts consisting of 64 threads each. A GPU can be used to render graphics, but today's GPUs are also fully programmable, allowing programmers to leverage the awesome computing power of a GPU to execute *compute shaders*. This is known as *general-purpose GPU* computing (GPGPU).

Virtually all GPUs break the graphics pipeline into the substages described below and depicted in Figure 11.44. Each stage is shaded to indicate whether its functionality is programmable, fixed but configurable, or fixed and non-configurable.

#### 11.2.4.1 Vertex Shader

This stage is fully programmable. It is responsible for transformation and shading/lighting of individual vertices. The input to this stage is a single vertex (although in practice many vertices are processed in parallel). Its position and normal are typically expressed in model space or world space. The vertex shader handles transformation from model space to view space via the model-view transform. Perspective projection is also applied, as well as per-vertex lighting and texturing calculations, and skinning for animated characters. The vertex shader can also perform procedural animation by modifying the position of the vertex. Examples of this include foliage that sways in the breeze or an undulating water surface. The output of this stage is a fully transformed and lit vertex, whose position and normal are expressed in homogeneous clip

space (see Section 11.1.4.4).

On modern GPUs, the vertex shader has full access to texture data—a capability that used to be available only to the pixel shader. This is particularly useful when textures are used as stand-alone data structures like heightmaps or look-up tables.

#### 11.2.4.2 Geometry Shader

This optional stage is also fully programmable. The geometry shader operates on entire primitives (triangles, lines and points) in homogeneous clip space. It is capable of culling or modifying input primitives, and it can also generate new primitives. Typical uses include shadow volume extrusion (see Section 11.3.3.1), rendering the six faces of a cube map (see Section 11.3.1.4), fur fin extrusion around silhouette edges of meshes, creation of particle quads from point data (see Section 11.4.1), dynamic tessellation, fractal subdivision of line segments for lightning effects, cloth simulations, and the list goes on.

#### 11.2.4.3 Stream Output

Some GPUs permit the data that has been processed up to this point in the pipeline to be written back to memory. From there, it can then be looped back to the top of the pipeline for further processing. This feature is called *stream output*.

Stream output permits a number of intriguing visual effects to be achieved without the aid of the CPU. An excellent example is hair rendering. Hair is often represented as a collection of cubic spline curves. It used to be that hair physics simulation would be done on the CPU. The CPU would also tessellate the splines into line segments. Finally the GPU would render the segments.

With stream output, the GPU can do the physics simulation on the control points of the hair splines within the vertex shader. The geometry shader tessellates the splines, and the stream output feature is used to write the tessellated vertex data to memory. The line segments are then piped back into the top of the pipeline so they can be rendered.

#### 11.2.4.4 Clipping

The clipping stage chops off those portions of the triangles that straddle the frustum. Clipping is done by identifying vertices that lie outside the frustum and then finding the intersection of the triangle's edges with the planes of the frustum. These intersection points become new vertices that define one or more clipped triangles.

This stage is fixed in function, but it is somewhat configurable. For example, user-defined clipping planes can be added in addition to the frustum

planes. This stage can also be configured to cull triangles that lie entirely outside the frustum.

#### 11.2.4.5 Screen Mapping

Screen mapping simply scales and shifts the vertices from homogeneous clip space into screen space. This stage is entirely fixed and non-configurable.

#### 11.2.4.6 Triangle Set-up

During triangle set-up, the rasterization hardware is initialized for efficient conversion of the triangle into fragments. This stage is not configurable.

#### 11.2.4.7 Triangle Traversal

Each triangle is broken into fragments (i.e., rasterized) by the triangle traversal stage. Usually one fragment is generated for each pixel, although with certain antialiasing techniques, multiple fragments may be created per pixel (see Section 11.1.4.7). The triangle traversal stage also interpolates vertex attributes in order to generate per-fragment attributes for processing by the pixel shader. Perspective-correct interpolation is used where appropriate. This stage's functionality is fixed and not configurable.

#### 11.2.4.8 Early z-Test

Many graphics cards are capable of checking the depth of the fragment at this point in the pipeline, discarding it if it is being occluded by the pixel already in the frame buffer. This allows the (potentially very expensive) pixel shader stage to be skipped entirely for occluded fragments.

Surprisingly, not all graphics hardware supports depth testing at this stage of the pipeline. In older GPU designs, the z-test was done along with alpha testing, after the pixel shader had run. For this reason, this stage is called the *early z-test* or *early depth test* stage.

#### 11.2.4.9 Pixel Shader

This stage is fully programmable. Its job is to shade (i.e., light and otherwise process) each fragment. The pixel shader can also discard fragments, for example because they are deemed to be entirely transparent. The pixel shader can address one or more texture maps, run per-pixel lighting calculations, and do whatever else is necessary to determine the fragment's color.

The input to this stage is a collection of per-fragment attributes (which have been interpolated from the vertex attributes by the triangle traversal stage). The output is a single color vector describing the desired color of the fragment.

#### II.2.4.10 Merging / Raster Operations Stage

The final stage of the pipeline is known as the *merging stage* or *blending stage*, also known as the *raster operations* stage or ROP in NVIDIA parlance. This stage is not programmable, but it is highly configurable. It is responsible for running various fragment tests including the depth test (see Section 11.1.4.8), alpha test (in which the values of the fragment's and pixel's alpha channels can be used to reject certain fragments) and stencil test (see Section 11.3.3.1).

If the fragment passes all of the tests, its color is blended (merged) with the color that is already present in the frame buffer. The way in which blending occurs is controlled by the *alpha blending function*—a function whose basic structure is hard-wired, but whose operators and parameters can be configured in order to produce a wide variety of blending operations.

Alpha blending is most commonly used to render semitransparent geometry. In this case, the following blending function is used:

$$\mathbf{C}'_D = A_S \mathbf{C}_S + (1 - A_S) \mathbf{C}_D.$$

The subscripts *S* and *D* stand for “source” (the incoming fragment) and “destination” (the pixel in the frame buffer), respectively. Therefore, the color that is written into the frame buffer ( $\mathbf{C}'_D$ ) is a *weighted average* of the existing frame buffer contents ( $\mathbf{C}_D$ ) and the color of the fragment being drawn ( $\mathbf{C}_S$ ). The blend weight ( $A_S$ ) is just the source alpha of the incoming fragment.

For alpha blending to look right, the semitransparent and translucent surfaces in the scene must be sorted and rendered in back-to-front order, *after* the opaque geometry has been rendered to the frame buffer. This is because after alpha blending has been performed, the depth of the new fragment *overwrites* the depth of the pixel with which it was blended. In other words, the depth buffer ignores transparency (unless depth writes have been turned off, of course). If we are rendering a stack of translucent objects on top of an opaque backdrop, the resulting pixel color should ideally be a blend between the opaque surface's color and the colors of *all* of the translucent surfaces in the stack. If we try to render the stack in any order other than back-to-front, depth-test failures will cause some of the translucent fragments to be discarded, resulting in an incomplete blend (and a rather odd-looking image).

Other alpha blending functions can be defined as well, for purposes other than transparency blending. The general blending equation takes the form  $\mathbf{C}'_D = (\mathbf{w}_S \otimes \mathbf{C}_S) + (\mathbf{w}_D \otimes \mathbf{C}_D)$ , where the weighting factors  $\mathbf{w}_S$  and  $\mathbf{w}_D$  can be selected by the programmer from a predefined set of values including zero,

one, source or destination color, source or destination alpha and one minus the source or destination color or alpha. The operator  $\otimes$  is either a regular scalar-vector multiplication or a component-wise vector-vector multiplication (a Hadamard product—see Section 5.2.4.1) depending on the data types of  $\mathbf{w}_S$  and  $\mathbf{w}_D$ .

### 11.2.5 Programmable Shaders

Now that we have an end-to-end picture of the GPU pipeline in mind, let's take a deeper look at the most interesting part of the pipeline—the programmable shaders. Shader architectures have evolved significantly since their introduction with DirectX 8. Early *shader models* supported only low-level assembly language programming, and the instruction set and register set of the pixel shader differed significantly from those of the vertex shader. DirectX 9 brought with it support for high-level C-like shader languages such as Cg (C for graphics), HLSL (High-Level Shading Language—Microsoft's implementation of the Cg language) and GLSL (OpenGL shading language). With DirectX 10, the geometry shader was introduced, and with it came a unified shader architecture called *shader model 4.0* in DirectX parlance. In the unified shader model, all three types of shaders support roughly the same instruction set and have roughly the same set of capabilities, including the ability to read texture memory.

A shader takes a single element of input data and transforms it into zero or more elements of output data.

- In the case of the vertex shader, the input is a vertex whose position and normal are expressed in model space or world space. The output of the vertex shader is a fully transformed and lit vertex, expressed in homogeneous clip space.
- The input to the geometry shader is a single  $n$ -vertex primitive—a point ( $n = 1$ ), line segment ( $n = 2$ ) or triangle ( $n = 3$ )—with up to  $n$  additional vertices that act as control points. The output is zero or more primitives, possibly of a different type than the input. For example, the geometry shader could convert points into two-triangle quads, or it could transform triangles into triangles but optionally discard some triangles and so on.
- The pixel shader's input is a fragment whose attributes have been interpolated from the three vertices of the triangle from which it came. The output of the pixel shader is the color that will be written into the frame buffer (presuming the fragment passes the depth test and other optional

tests). The pixel shader is also capable of discarding fragments explicitly, in which case it produces no output.

### II.2.5.1 Accessing Memory

Because the GPU implements a data processing pipeline, access to RAM is carefully controlled. A shader program usually cannot read from or write to memory directly. Instead, its memory accesses are limited to two methods: registers and texture maps.

However, we should note that these restrictions are lifted on systems in which the GPU and CPU share memory directly. For example, the AMD Jaguar system on a chip (SoC) that sits at the heart of the PlayStation 4 is an example of a *heterogeneous system architecture* (HSA). On a non-HSA system, the CPU and GPU are typically separate devices, each with its own private memory, and each usually residing on a separate circuit board. Transferring data between the two processors requires cumbersome, high-latency communication over a specialized bus such as AGP or PCIe. With HSA, the CPU and GPU share a single unified memory store called a *heterogeneous unified memory architecture* (hUMA). Shaders running on a system with hUMA, like the PS4, can therefore be passed a *shader resource table* (SRT) as input. This is just a pointer to a C/C++ struct in memory that can be read from or written to by both the CPU and the shader running on the GPU. On the PS4, SRTs take the place of the constant registers described in the following sections.

#### *Shader Registers*

A shader can access RAM indirectly via *registers*. All GPU registers are in 128-bit SIMD format. Each register is capable of holding four 32-bit floating-point or integer values (represented by the `float4` data type in the Cg language). Such a register can contain a four-element vector in homogeneous coordinates or a color in RGBA format, with each component in 32-bit floating-point format. Matrices can be represented by groups of three or four registers (represented by built-in matrix types like `float4x4` in Cg). A GPU register can also be used to hold a single 32-bit scalar, in which case the value is usually replicated across all four 32-bit fields. Some GPUs can operate on 16-bit fields, known as *halfs*. (Cg provides various built-in types like `half4` and `half4x4` for this purpose.)

Registers come in four flavors, as follows:

- *Input registers.* These registers are the shader's primary source of input data. In a vertex shader, the input registers contain attribute data



obtained directly from the vertices. In a pixel shader, the input registers contain interpolated vertex attribute data corresponding to a single fragment. The values of all input registers are set automatically by the GPU prior to invoking the shader.

- *Constant registers.* The values of constant registers are set by the application and can change from primitive to primitive. Their values are constant only from the point of view of the shader program. They provide a secondary form of input to the shader. Typical contents include the model-view matrix, the projection matrix, light parameters and any other parameters required by the shader that are not available as vertex attributes.
- *Temporary registers.* These registers are for use by the shader program internally and are typically used to store intermediate results of calculations.
- *Output registers.* The contents of these registers are filled in by the shader and serve as its only form of output. In a vertex shader, the output registers contain vertex attributes such as the transformed position and normal vectors in homogeneous clip space, optional vertex colors, texture coordinates and so on. In a pixel shader, the output register contains the final color of the fragment being shaded.

The application provides the values of the constant registers when it submits primitives for rendering. The GPU automatically copies vertex or fragment attribute data from video RAM into the appropriate input registers prior to calling the shader program, and it also writes the contents of the output registers back into RAM at the conclusion of the program's execution so that the data can be passed to the next stage of the pipeline.

GPUs typically cache output data so that it can be reused without being recalculated. For example, the *post-transform vertex cache* stores the most-recently processed vertices emitted by the vertex shader. If a triangle is encountered that refers to a previously processed vertex, it will be read from the post-transform vertex cache if possible—the vertex shader need only be called again if the vertex in question has since been ejected from the cache to make room for newly processed vertices.

### Textures

A shader also has direct read-only access to *texture maps*. Texture data is addressed via texture coordinates, rather than via absolute memory addresses. The GPU's texture samplers automatically *filter* the texture data, blending values between adjacent texels or adjacent mipmap levels as appropriate. Texture

filtering can be disabled in order to gain direct access to the values of particular texels. This can be useful when a texture map is used as a data table, for example.

Shaders can only *write* to texture maps in an indirect manner—by rendering the scene to an off-screen frame buffer that is interpreted as a texture map by subsequent rendering passes. This feature is known as *render to texture*.

### II.2.5.2 Introduction to High-Level Shader Language Syntax

High-level shader languages like Cg and GLSL are modeled after the C programming language. The programmer can declare functions, define a simple struct, and perform arithmetic. However, as we said above, a shader program only has access to registers and textures. As such, the struct and variable we declare in Cg or GLSL is mapped directly onto registers by the shader compiler. We define these mappings in the following ways:

- *Semantics*. Variables and struct members can be suffixed with a colon followed by a keyword known as a *semantic*. The semantic tells the shader compiler to bind the variable or data member to a particular vertex or fragment attribute. For example, in a vertex shader we might declare an input struct whose members map to the *position* and *color* attributes of a vertex as follows:

```
struct VtxOut
{
    float4 pos : POSITION; // map to position attribute
    float4 color : COLOR; // map to color attribute
};
```

- *Input versus output*. The compiler determines whether a particular variable or struct should map to input or output registers from the context in which it is used. If a variable is passed as an argument to the shader program's main function, it is assumed to be an input; if it is the return value of the main function, it is taken to be an output.

```
VtxOut vshaderMain(VtxIn in) // maps to input registers
{
    VtxOut out;
    // ...
    return out; // maps to output registers
}
```

- *Uniform declaration*. To gain access to the data supplied by the application via the constant registers, we can declare a variable with the keyword

uniform. For example, the model-view matrix could be passed to a vertex shader as follows:

```
VtxOut vshaderMain(
    VtxIn in,
    uniform float4x4 modelViewMatrix)
{
    VtxOut out;
    // ...
    return out;
}
```

Arithmetic operations can be performed by invoking C-style operators, or by calling intrinsic functions as appropriate. For example, to multiply the input vertex position by the model-view matrix, we could write:

```
VtxOut vshaderMain(VtxIn in,
    uniform float4x4 modelViewMatrix)
{
    VtxOut out;

    out.pos = mul(modelViewMatrix, in.pos);
    out.color = float4(0, 1, 0, 1); // RGBA green

    return out;
}
```

Data is obtained from textures by calling special intrinsic functions that read the value of the texels at a specified texture coordinate. A number of variants are available for reading one-, two- and three-dimensional textures in various formats, with and without filtering. Special texture addressing modes are also available for accessing cube maps and shadow maps. References to the texture maps themselves are declared using a special data type known as a *texture sampler* declaration. For example, the data type `sampler2D` represents a reference to a typical two-dimensional texture. The following simple Cg pixel shader applies a diffuse texture to a triangle:

```
struct FragmentOut
{
    float4 color : COLOR;
};

FragmentOut pshaderMain(float2 uv : TEXCOORD0,
    uniform sampler2D texture)
{
    FragmentOut out;
```

```

        // look up texel at (u,v)
        out.color = tex2D(texture, uv);

    return out;
}

```

### II.2.5.3 Effect Files

By itself, a shader program isn't particularly useful. Additional information is required by the GPU pipeline in order to call the shader program with meaningful inputs. For example, we need to specify how the application-specified parameters, like the model-view matrix, light parameters and so on, map to the uniform variables declared in the shader program. In addition, some visual effects require two or more rendering passes, but a shader program only describes the operations to be applied during a single rendering pass. If we are writing a game for the PC platform, we will need to define "fallback" versions of some of our more-advanced rendering effects, so that they will work even on older graphics cards. To tie our shader program(s) together into a complete visual effect, we turn to a file format known as an *effect file*.

Different rendering engines implement effects in slightly different ways. In Cg, the effect file format is known as CgFX. OGRE uses a file format very similar to CgFX known as a *material file*. GLSL effects can be described using the COLLADA format, which is based on XML. Despite the differences, effects generally take on the following hierarchical format:

- At global scope, structs, shader programs (implemented as various "main" functions) and global variables (which map to application-specified constant parameters) are defined.
- One or more *techniques* are defined. A technique represents one way to render a particular visual effect. An effect typically provides a primary technique for its highest-quality implementation and possibly a number of fallback techniques for use on lower-powered graphics hardware.
- Within each technique, one or more *passes* are defined. A pass describes how a single full-frame image should be rendered. It typically includes a reference to a vertex, geometry and/or pixel shader program's "main" function, various parameter bindings and optional render state settings.

### II.2.5.4 Further Reading

In this section, we've only had a small taste of what high-level shader programming is like—a complete tutorial is beyond our scope here. For a much

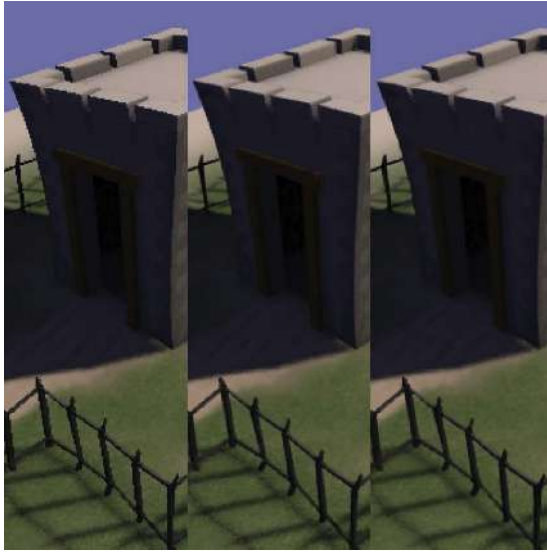


Figure 11.45. No antialiasing (left), 4× MSAA (center) and Nvidia's FXAA, preset 3 (right). Image from Nvidia's FXAA white paper by Timothy Lottes (<http://bit.ly/1mlzCTv>). (See Color Plate XIX.)

more detailed introduction to Cg shader programming, refer to the Cg tutorial available on NVIDIA's website at <https://developer.nvidia.com/content/hello-cg-introductory-tutorial>.

### 11.2.6 Antialiasing

When a triangle is rasterized, its edges can look jagged—the familiar “stair step” effect we have all come to know and love (or hate). Technically speaking, aliasing arises because we are using a discrete set of pixels to *sample* an image that is really a smooth, continuous two-dimensional signal. (See Section 14.3.2.1 for a detailed discussion of sampling and aliasing.)

The term *antialiasing* describes any technique that reduces the visual artifacts caused by aliasing. There are many different ways to antialias a rendered scene. The net effect of pretty much all of them is to “soften” the edges of rendered triangles by blending them with surrounding pixels. Each technique has unique performance, memory-usage and quality characteristics. Figure 11.45 shows a scene rendered first without antialiasing, then with 4× MSAA and finally with Nvidia's FXAA technique.

### II.2.6.1 Full-Screen Antialiasing (FSAA)

In this technique, also known as *super-sampled antialiasing* (SSAA), the scene is rendered into a frame buffer that is larger than the actual screen. Once rendering of the frame is complete, the resulting oversized image is *downsampled* to the desired resolution. In  $4\times$  supersampling, the rendered image is twice as wide and twice as tall as the screen, resulting in a frame buffer that occupies four times the memory. It also requires four times the GPU processing power because the pixel shader must be run four times for each screen pixel. As you can see, FSAA is an incredibly expensive technique both in terms of memory consumption and GPU cycles. As such, it is rarely used in practice.

### II.2.6.2 Multisampled Antialiasing (MSAA)

Multisampled antialiasing is a technique that provides visual quality comparable to that of FSAA, while consuming a great deal less GPU bandwidth (and the same amount of video RAM). The MSAA approach is based on the observation that, thanks to the natural antialiasing effect of texture *mipmapping*, aliasing tends to be a problem primarily at the *edges* of triangles, not in their interiors.

To understand how MSAA works, recall that the process of rasterizing a triangle really boils down to three distinct operations: (1) Determining which pixels the triangle overlaps (coverage), (2) determining whether or not each pixel is occluded by some other triangle (depth testing) and (3) determining the color of each pixel, presuming that the coverage and depth tests tell us that the pixel should in fact be drawn (pixel shading).

When rasterizing a triangle without antialiasing, the coverage test, depth test and pixel shading operations are all run at a single idealized point within each screen pixel, usually located at its center. In MSAA, the coverage and depth tests are run for  $N$  points known as *subsamples* within each screen pixel.  $N$  is typically chosen to be 2, 4, 5, 8 or 16. However, the pixel shader is only run *once* per screen pixel, no matter how many subsamples we use. This gives MSAA a big advantage over FSAA in terms of GPU bandwidth, because shading is typically a great deal more expensive than coverage and depth testing.

In  $N\times$  MSAA, the depth, stencil and color buffers are each allocated to be  $N$  times as large as they would otherwise be. For each screen pixel, these buffers contain  $N$  “slots,” one slot for each subsample. When rasterizing a triangle, the coverage and depth tests are run  $N$  times for the  $N$  subsamples within each fragment of the triangle. If at least one of the  $N$  tests indicates that the fragment should be drawn, the pixel shader is run once. The color obtained from the pixel shader is then stored *only* into those slots that correspond to

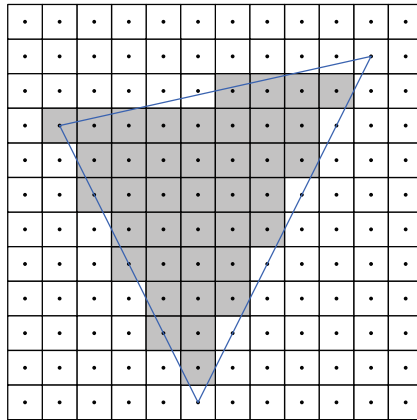


Figure 11.46. Rasterizing a triangle without antialiasing.

the subsamples that fell *inside* the triangle. Once the entire scene has been rendered, the oversized color buffer is downsampled to yield the final screen-resolution image. This process involves averaging the color values found in the  $N$  subsample slots for each screen pixel. The net result is an antialiased image with a shading cost equal to that of a non-antialiased image.

In Figure 11.46 we see a triangle that has been rasterized without antialiasing. Figure 11.47 illustrates the  $4\times$  MSAA technique. For more information on MSAA, see <http://mynameismjp.wordpress.com/2012/10/24/msaa-overview>.

### 11.2.6.3 Coverage Sample Antialiasing (CSAA)

This technique is an optimization of the MSAA technique pioneered by Nvidia. For  $4\times$  CSAA, the pixel shader is run once, the depth test and color storage is done for four subsample points per fragment, but the pixel coverage test is performed for 16 “coverage subsamples” per fragment. This produces finer-grained color blending at the edges of triangles, similar to what you’d see with  $8\times$  or  $16\times$  MSAA, but at the memory and GPU cost of  $4\times$  MSAA.

### 11.2.6.4 Morphological Antialiasing (MLAA)

Morphological antialiasing focuses its efforts on correcting only those regions of a scene that suffer the most from the effects of aliasing. In MLAA, the scene is rendered at normal size, and then scanned in order to identify stair-stepped patterns. When these patterns are found, they are blurred to reduce the effects of aliasing. Fast approximate antialiasing (FXAA) is an optimized technique

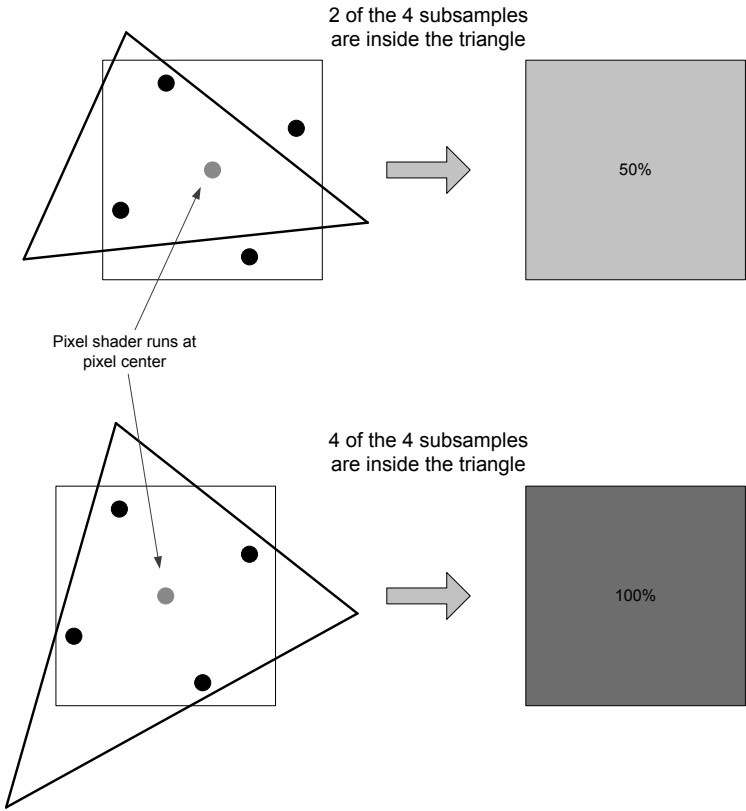


Figure II.47. Multisampled antialiasing (MSAA).



developed by Nvidia that is similar to MLAA in its approach.

For a detailed discussion of MLAA, see <https://intel.ly/2HhrQWX>. FXAA is described in detail here: <https://bit.ly/1mIzCTv>.

#### 11.2.6.5 Subpixel Morphological Antialiasing (SMAA)

Subpixel Morphological Antialiasing (SMAA) combines morphological antialiasing (MLAA and FXAA) techniques with multisampling/supersampling strategies (MSAA, SSAA) to produce more accurate subpixel features. Like FXAA, it's an inexpensive technique, but it blurs the final image less than FXAA. For these reasons, it's arguably the best AA solution available today. A detailed coverage of this topic is beyond our scope in this book, but you can read more about SMAA at <http://www.iryoku.com/smaa/>.

### 11.2.7 The Application Stage

Now that we understand how the GPU works, we can discuss the pipeline stage that is responsible for driving it—the application stage. This stage has three roles:

1. *Visibility determination.* Only objects that are visible (or at least *potentially* visible) should be submitted to the GPU, lest we waste valuable resources processing triangles that will never be seen.
2. *Submitting geometry to the GPU for rendering.* Submesh-material pairs are sent to the GPU via a rendering call like `DrawIndexedPrimitive()` (DirectX) or `glDrawArrays()` (OpenGL), or via direct construction of the GPU command list. The geometry may be sorted for optimal rendering performance. Geometry might be submitted more than once if the scene needs to be rendered in multiple passes.
3. *Controlling shader parameters and render state.* The uniform parameters passed to the shader via constant registers are configured by the application stage on a per-primitive basis. In addition, the application stage must set all of the configurable parameters of the non-programmable pipeline stages to ensure that each primitive is rendered appropriately.

In the following sections, we'll briefly explore how the application stage performs these tasks.

#### 11.2.7.1 Visibility Determination

The cheapest triangles are the ones you never draw. So it's incredibly important to *cull* objects from the scene that do not contribute to the final rendered

image prior to submitting them to the GPU. The process of constructing the list of visible mesh instances is known as *visibility determination*.

### *Frustum Culling*

In frustum culling, all objects that lie entirely outside the frustum are excluded from our render list. Given a candidate mesh instance, we can determine whether or not it lies inside the frustum by performing some simple tests between the object's *bounding volume* and the six frustum planes. The bounding volume is usually a sphere, because spheres are particularly easy to cull. For each frustum plane, we move the plane outward a distance equal to the radius of the sphere, then we determine on which side of each modified plane the center point of the sphere lies. If the sphere is found to be on the front side of all six modified planes, the sphere is inside the frustum.

In practice, we don't need to actually move the frustum planes. Recall from Equation (5.13) that the perpendicular distance  $h$  from a point to a plane can be calculated by plugging the point directly into the plane equation as follows:  $h = ax + by + cz + d = \mathbf{n} \cdot \mathbf{P} - \mathbf{n} \cdot \mathbf{P}_0$  (see Section 5.6.3). So all we need to do is plug the center point of our bounding sphere into the plane equations for each frustum plane, giving us a value of  $h_i$  for each plane  $i$ , and then we can compare the  $h_i$  values to the radius of the bounding sphere to determine whether or not it lies inside each plane.

A scene graph data structure, described in Section 11.2.7.4, can help optimize frustum culling by allowing us to ignore objects whose bounding spheres are nowhere close to being inside the frustum.

### *Occlusion and Potentially Visible Sets*

Even when objects lie entirely within the frustum, they may occlude one another. Removing objects from the visible list that are entirely occluded by other objects is called *occlusion culling*. In crowded environments viewed from ground level, there can be a great deal of inter-object occlusion, making occlusion culling extremely important. In less crowded scenes, or when scenes are viewed from above, much less occlusion may be present, and the cost of occlusion culling may outweigh its benefits.

Gross occlusion culling of a large-scale environment can be done by precalculating a *potentially visible set* (PVS). For any given camera vantage point, a PVS lists those scene objects that might be visible. A PVS errs on the side of including objects that aren't actually visible, rather than excluding objects that actually would have contributed to the rendered scene.

One way to implement a PVS system is to chop the level up into regions of some kind. Each region can be provided with a list of the other regions

that can be seen when the camera is inside it. These PVSs might be manually specified by the artists or game designers. More commonly, an automated offline tool generates the PVS based on user-specified regions. Such a tool usually operates by rendering the scene from various randomly distributed vantage points within a region. Every region's geometry is color coded, so the list of visible regions can be found by scanning the resulting frame buffer and tabulating the region colors that are found. Because automated PVS tools are imperfect, they typically provide the user with a mechanism for tweaking the results, either by manually placing vantage points for testing, or by manually specifying a list of regions that should be explicitly included or excluded from a particular region's PVS.

### *Portals*

Another way to determine what portions of a scene are visible is to use *portals*. In portal rendering, the game world is divided up into semiclosed regions that are connected to one another via holes, such as windows and doorways. These holes are called portals. They are usually represented by polygons that describe their boundaries.

To render a scene with portals, we start by rendering the region that contains the camera. Then, for each portal in the region, we extend a frustum-like volume consisting of planes extending from the camera's focal point through each edge of the portal's bounding polygon. The contents of the neighboring region can be culled to this portal volume in exactly the same way geometry is culled against the camera frustum. This ensures that only the visible geometry in the adjacent regions will be rendered. Figure 11.48 provides an illustration of this technique.

### *Occlusion Volumes (Antiportals)*

If we flip the portal concept on its head, pyramidal volumes can also be used to describe regions of the scene that *cannot* be seen because they are being occluded by an object. These volumes are known as *occlusion volumes* or *antiportals*. To construct an occlusion volume, we find the silhouette edges of each occluding object and extend planes outward from the camera's focal point through each of these edges. We test more-distant objects against these occlusion volumes and cull them if they lie entirely within the occlusion region. This is illustrated in Figure 11.49.

Portals are best used when rendering enclosed indoor environments with a relatively small number of windows and doorways between "rooms." In this kind of scene, the portals occupy a relatively small percentage of the total

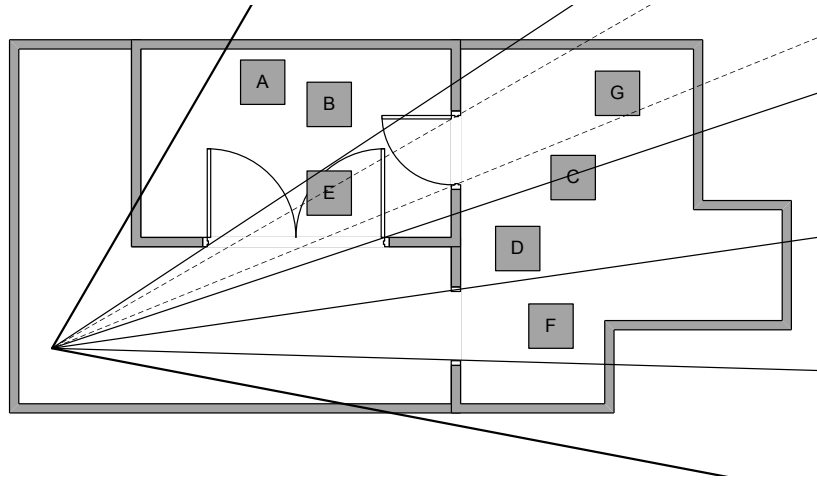


Figure 11.48. Portals are used to define frustum-like volumes, which are used to cull the contents of neighboring regions. In this example, objects A, B and D will be culled because they lie outside one of the portals; the other objects will be visible.

volume of the camera frustum, resulting in a large number of objects outside the portals that can be culled. Antiportals are best applied to large outdoor environments, in which nearby objects often occlude large swaths of the camera frustum. In this case, the antiportals occupy a relatively large percentage of the total camera frustum volume, resulting in large numbers of culled objects.

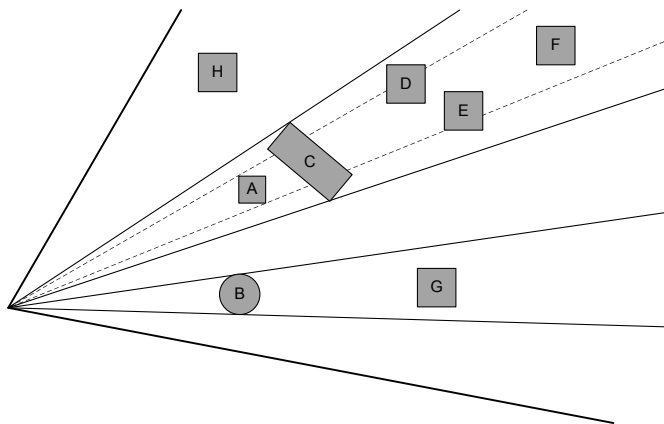


Figure 11.49. As a result of the antiportals corresponding to objects A, B and C, objects D, E, F and G are culled. Therefore, only A, B, C and H are visible.

### 11.2.7.2 Primitive Submission

Once a list of visible geometric primitives has been generated, the individual primitives must be *submitted* to the GPU pipeline for rendering. This can be accomplished by making calls to `DrawIndexedPrimitive()` in DirectX or `glDrawArrays()` in OpenGL.

#### *Render State*

As we learned in Section 11.2.4, the functionality of many of the GPU pipeline's stages is fixed but configurable. And even programmable stages are driven in part by configurable parameters. Some examples of these configurable parameters are listed below (although this is by no means a complete list):

- world-view matrix;
- light direction vectors;
- texture bindings (i.e., which textures to use for a given material/shader);
- texture addressing and filtering modes;
- time base for scrolling textures and other animated effects;
- z-test (enabled or disabled); and
- alpha blending options.

The set of all configurable parameters within the GPU pipeline is known as the *hardware state* or *render state*. It is the application stage's responsibility to ensure that the hardware state is configured properly and completely for each submitted primitive. Ideally these state settings are described completely by the material associated with each submesh. So the application stage's job boils down to iterating through the list of visible mesh instances, iterating over each submesh-material pair, setting the render state based on the material's specifications and then calling the low-level primitive submission functions (`DrawIndexedPrimitive()`, `glDrawArrays()`, or similar).

#### *State Leaks*

If we forget to set some aspect of the render state between submitted primitives, the settings used on the previous primitive will “leak” over onto the new primitive. A *render state leak* might manifest itself as an object with the wrong texture or an incorrect lighting effect, for example. Clearly it's important that the application stage never allow state leaks to occur.

### *The GPU Command List*

The application stage actually communicates with the GPU via a command list. These commands interleave render state settings with references to the geometry that should be drawn. For example, to render objects A and B with material 1, followed by objects C, D and E using material 2, the command list might look like this:

- Set render state for material 1 (multiple commands, one per render state setting).
- Submit primitive A.
- Submit primitive B.
- Set render state for material 2 (multiple commands).
- Submit primitive C.
- Submit primitive D.
- Submit primitive E.

Under the hood, API functions like `DrawIndexedPrimitive()` actually just construct and submit GPU command lists. The cost of these API calls can themselves be too high for some applications. To maximize performance, some game engines build GPU command lists manually or by calling a low-level rendering API like Vulkan (<https://www.khronos.org/vulkan/>).

#### II.2.7.3 Geometry Sorting

Render state settings are global—they apply to the entire GPU as a whole. So in order to change render state settings, the entire GPU pipeline must be flushed before the new settings can be applied. This can cause massive performance degradation if not managed carefully.

Clearly we'd like to change render settings as infrequently as possible. The best way to accomplish this is to sort our geometry by material. That way, we can install material A's settings, render all geometry associated with material A and then move on to material B.

Unfortunately, sorting geometry by material can have a detrimental effect on rendering performance because it increases *overdraw*—a situation in which the same pixel is filled multiple times by multiple overlapping triangles. Certainly some overdraw is necessary and desirable, as it is the only way to properly alpha-blend transparent and translucent surfaces into a scene. However, overdraw of *opaque* pixels is always a waste of GPU bandwidth.

The early z-test is designed to discard occluded fragments before the expensive pixel shader has a chance to execute. But to take maximum advantage of early z, we need to draw the triangles in front-to-back order. That way, the

closest triangles will fill the z-buffer right off the bat, and all of the fragments coming from more-distant triangles behind them can be quickly discarded, with little or no overdraw.

### *z-Prepass to the Rescue*

How can we reconcile the need to sort geometry by material with the conflicting need to render opaque geometry in a front-to-back order? The answer lies in a GPU feature known as *z-prepass*.

The idea behind *z-prepass* is to render the scene twice: the first time to generate the contents of the z-buffer as efficiently as possible and the second time to populate the frame buffer with full color information (but this time with no overdraw, thanks to the contents of the z-buffer). The GPU provides a special double-speed rendering mode in which the pixel shaders are disabled, and only the z-buffer is updated. Opaque geometry can be rendered in front-to-back order during this phase, to minimize the time required to generate the z-buffer contents. Then the geometry can be resorted into material order and rendered in full color with minimal state changes for maximum pipeline throughput.

Once the opaque geometry has been rendered, transparent surfaces can be drawn in back-to-front order. This brute-force method allows us to achieve the proper alpha-blended result. *Order-independent transparency* (OIT) is a technique that permits transparent geometry to be rendered in an arbitrary order. It works by storing multiple fragments per pixel, sorting each pixel's fragments and blending them only after the entire scene has been rendered. This technique produces correct results without the need for pre-sorting the geometry, but it comes at a high memory cost because the frame buffer must be large enough to store all of the translucent fragments for each pixel.

#### 11.2.7.4 Scene Graphs

Modern game worlds can be very large. The majority of the geometry in most scenes does not lie within the camera frustum, so frustum culling all of these objects explicitly is usually incredibly wasteful. Instead, we would like to devise a data structure that manages all of the geometry in the scene and allows us to quickly discard large swaths of the world that are nowhere near the camera frustum prior to performing detailed frustum culling. Ideally, this data structure should also help us to sort the geometry in the scene, either in front-to-back order for the *z-prepass* or in material order for full-color rendering.

Such a data structure is often called a *scene graph*, in reference to the graph-like data structures often used by film rendering engines and DCC tools like Maya. However, a game's scene graph needn't actually be a graph, and in fact the data structure of choice is usually some kind of tree (which is, of course,

a special case of a graph). The basic idea behind most of these data structures is to partition three-dimensional space in a way that makes it easy to discard regions that do not intersect the frustum, without having to frustum cull all of the individual objects within them. Examples include quadtrees and octrees, BSP trees, *kd*-trees and spatial hashing techniques.

### *Quadtrees and Octrees*

A *quadtree* divides space into quadrants recursively. Each level of recursion is represented by a node in the quadtree with four children, one for each quadrant. The quadrants are typically separated by vertically oriented, axis-aligned planes, so that the quadrants are square or rectangular. However, some quadtrees subdivide space using arbitrarily shaped regions.

Quadrees can be used to store and organize virtually any kind of spatially distributed data. In the context of rendering engines, quadrees are often used to store renderable primitives such as mesh instances, subregions of terrain geometry or individual triangles of a large static mesh, for the purposes of efficient frustum culling. The renderable primitives are stored at the leaves of the tree, and we usually aim to achieve a roughly uniform number of primitives within each leaf region. This can be achieved by deciding whether to continue or terminate the subdivision based on the number of primitives within a region.

To determine which primitives are visible within the camera frustum, we walk the tree from the root to the leaves, checking each region for intersection with the frustum. If a given quadrant does not intersect the frustum, then we know that none of its child regions will do so either, and we can stop traversing that branch of the tree. This allows us to search for potentially visible primitives much more quickly than would be possible with a linear search (usually in  $O(\log n)$  time). An example of a quadtree subdivision of space is shown in Figure 11.50.

An *octree* is the three-dimensional equivalent of a quadtree, dividing space into eight subregions at each level of the recursive subdivision. The regions of an octree are often cubes or rectangular prisms but can be arbitrarily shaped three-dimensional regions in general.

### *Bounding Sphere Trees*

In the same way that a quadtree or octree subdivides space into (usually) rectangular regions, a *bounding sphere tree* divides space into spherical regions hierarchically. The leaves of the tree contain the bounding spheres of the renderable primitives in the scene. We collect these primitives into small logical



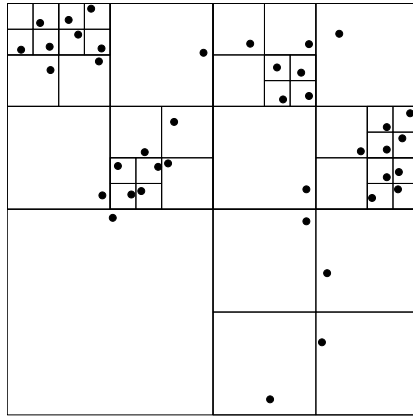


Figure 11.50. A top-down view of a space divided recursively into quadrants for storage in a quadtree, based on the criterion of one point per region.

groups and calculate the net bounding sphere of each group. The groups are themselves collected into larger groups, and this process continues until we have a single group with a bounding sphere that encompasses the entire virtual world. To generate a list of potentially visible primitives, we walk the tree from the root to the leaves, testing each bounding sphere against the frustum, and only recursing down branches that intersect it.

### *BSP Trees*

A binary space partitioning (BSP) tree divides space in half recursively until the objects within each half-space meet some predefined criteria (much as a quadtree divides space into quadrants). BSP trees have numerous uses, including collision detection and constructive solid geometry, as well as their most well-known application as a method for increasing the performance of frustum culling and geometry sorting for 3D graphics. A *kd-tree* is a generalization of the BSP tree concept to  $k$  dimensions.

In the context of rendering, a BSP tree divides space with a single plane at each level of the recursion. The dividing planes can be axis-aligned, but more commonly each subdivision corresponds to the plane of a single triangle in the scene. All of the other triangles are then categorized as being either on the front side or the back side of the plane. Any triangles that intersect the dividing plane are themselves divided into three new triangles, so that every triangle lies either entirely in front of or entirely behind the plane, or is coplanar with it. The result is a binary tree with a dividing plane and one or more triangles at each interior node and triangles at the leaves.

A BSP tree can be used for frustum culling in much the same way a quad-tree, octree or bounding sphere tree can. However, when generated with individual triangles as described above, a BSP tree can also be used to sort triangles into a strictly back-to-front or front-to-back order. This was particularly important for early 3D games like *Doom*, which did not have the benefit of a z-buffer and so were forced to use the painter's algorithm (i.e., to render the scene from back to front) to ensure proper inter-triangle occlusion.

Given a camera view point in 3D space, a back-to-front sorting algorithm walks the tree from the root. At each node, we check whether the view point is in front of or behind that node's dividing plane. If the camera is in front of a node's plane, we visit the node's back children first, then draw any triangles that are coplanar with its dividing plane, and finally we visit its front children. Likewise, when the camera's view point is found to be behind a node's dividing plane, we visit the node's front children first, then draw the triangles coplanar with the node's plane and finally we visit its back children. This traversal scheme ensures that the triangles farthest from the camera will be visited before those that are closer to it, and hence it yields a back-to-front ordering. Because this algorithm traverses *all* of the triangles in the scene, the order of the traversal is independent of the direction the camera is looking. A secondary frustum culling step would be required in order to traverse only visible triangles. A simple BSP tree is shown in Figure 11.51, along with the tree traversal that would be done for the camera position shown.

Full coverage of BSP tree generation and usage algorithms is beyond our scope here. See <http://www.gamedev.net/reference/articles/article657.asp>

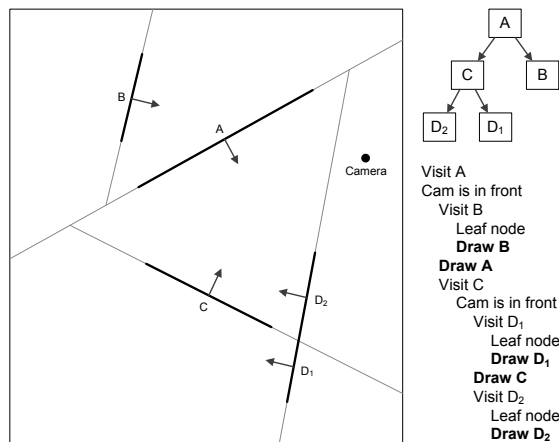


Figure 11.51. An example of back-to-front traversal of the triangles in a BSP tree. The triangles are shown edge-on in two dimensions for simplicity, but in a real BSP tree the triangles and dividing planes would be arbitrarily oriented in space.